Solve the Water Jug **SIMATS ENGINEERING**

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: **Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.** ***(BL3)***

Assessment Tool Weightage: 50%

QUESTIONS: 5 **Duration : 1 Hour**

Total Marks:5

Annexure A

Analytical Problem Solving

Code of Conduct:

I ____M.SHARMILA(192572139)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Analytical Problem Solving

1. **Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.**

2.Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.

i.What are the percept's for this agent?

ii.Characterize the operating environment.

iii.What are the actions the agent can take?

iv.How can one evaluate the performance of the agent?

v.What sort of agent architecture do you think is most suitable for this agent? Why?

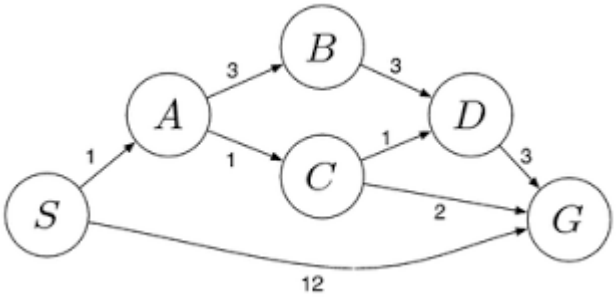
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost

path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process

Accuracy of Calculation	**20%**	**All calculations accurate; correct final answer**	**Minor calculation errors**	**Frequent errors; incorrect final answer**	**No valid calculations**
Interpretation of Results	**15%**	**Clearly interprets results with units and engineering relevance**	**Basic interpretation with minor omissions**	**Limited or unclear interpretation**	**No interpretation**
Problem Understanding	**20%**	**Clearly interprets problem, identifies all parameters and requirements accurately**	**Understands problem with minor gaps**	**Partial understanding; misses key elements**	**Misinterprets or unable to understand problem**

1.Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

1. Introduction

The Water Jug Problem is a classic Artificial Intelligence (AI) problem used to demonstrate state-space search and problem-solving techniques. The objective is to obtain a desired quantity of water by performing a sequence of valid operations on the available jugs.

Problem Statement

You are given:

- One 4-gallon jug
- One 3-gallon jug
- Neither jug has measurement markings.
- An unlimited water supply is available.

Allowed Operations

1. Fill any jug completely.
2. Empty any jug onto the ground.
3. Pour water from one jug into another until:
    - the source jug becomes empty, or
    - the destination jug becomes full.

Goal

Measure exactly 2 gallons of water in the 4-gallon jug.

2. Problem Formulation

A problem in Artificial Intelligence consists of the following components:

Initial State

Both jugs are empty.

State = (0,0)

where

- First value = Water in 4-gallon jug
- Second value = Water in 3-gallon jug

Example:

(4,3)

means

- 4-gallon jug contains 4 gallons
- 3-gallon jug contains 3 gallons

State Space

Every possible combination of water in both jugs forms a state.

Example states:

(0,0)

(4,0)

(1,3)

(2,3)

(2,0)

(4,2)

...

Operators (Actions)

The possible actions are:

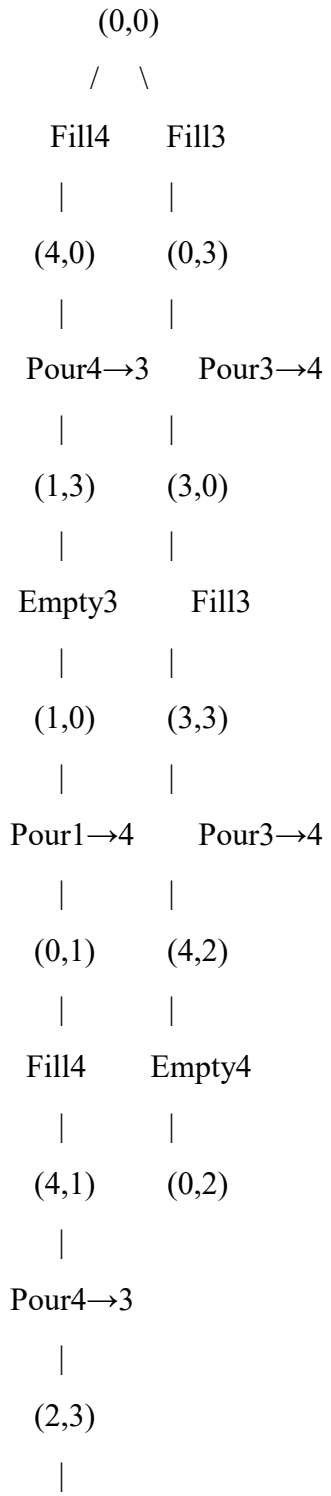
- Fill 4-gallon jug
- Fill 3-gallon jug
- Empty 4-gallon jug
- Empty 3-gallon jug
- Pour 4 → 3
- Pour 3 → 4

Goal State

(2,0)

The 4-gallon jug contains exactly **2 gallons**.

3. State Space Diagram (Simplified)



Empty3

|

(2,0) ← Goal

Code implementation

```
# Water Jug Problem (4 Gallon & 3 Gallon)

jug4 = 0
jug3 = 0

print("Initial State:", jug4, jug3)

# Fill 3-gallon jug
jug3 = 3
print("Fill 3-gallon jug:", jug4, jug3)

# Pour into 4-gallon jug
jug4 = jug3
jug3 = 0
print("Pour into 4-gallon jug:", jug4, jug3)

# Fill 3-gallon jug again
jug3 = 3
print("Fill 3-gallon jug:", jug4, jug3)

# Pour into 4-gallon jug until full
transfer = 4 - jug4
jug4 = 4
jug3 = jug3 - transfer
print("Pour again:", jug4, jug3)

# Empty 4-gallon jug
jug4 = 0
print("Empty 4-gallon jug:", jug4, jug3)

# Pour remaining water into 4-gallon jug
jug4 = jug3
jug3 = 0
print("Final State:", jug4, jug3)

if jug4 == 2:
    print("Goal Achieved!")
```

```
Python 3.14.2 (tags/v3.14.2:df79316, Dec  5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/M. Sharmila/OneDrive/Documents/ai.py ==========
Initial State: 0 0
Fill 3-gallon jug: 0 3
Pour into 4-gallon jug: 3 0
Fill 3-gallon jug: 3 3
Pour again: 4 2
Empty 4-gallon jug: 0 2
Final State: 2 0
Goal Achieved!
>>>
```

Implementation 1

4. Step-by-Step Solution

Step	Action	4-Gallon Jug	3-Gallon Jug
1	Initial state	0	0
2	Fill 3-gallon jug	0	3
3	Pour 3 → 4	3	0
4	Fill 3-gallon jug again	3	3
5	Pour 3 → 4 until 4-gallon jug is full	4	2
6	Empty 4-gallon jug	0	2
7	Pour remaining 2 gallons into 4-gallon jug	2	0

Goal Achieved

The 4-gallon jug contains exactly **2 gallons**.

5. Search Tree

(0,0)

/ \

(4,0) (0,3)

| |

(1,3) (3,0)

| |

(1,0) (3,3)

| |

(0,1) (4,2)

| |

(4,1) (0,2)

|

(2,3)

|

(2,0)

Goal State:

(2,0)

6. Algorithm

Algorithm: Water Jug Problem

Step 1: Start.

Step 2: Initialize both jugs as empty.

Step 3: Fill the 3-gallon jug.

Step 4: Pour water into the 4-gallon jug.

Step 5: Fill the 3-gallon jug again.

Step 6: Pour water into the 4-gallon jug until it becomes full.

Step 7: Empty the 4-gallon jug.

Step 8: Transfer the remaining 2 gallons from the 3-gallon jug to the 4-gallon jug.

Step 9: Stop.

7. Pseudocode

START

Jug4 = 0

Jug3 = 0

Fill(Jug3)

Pour(Jug3 → Jug4)

Fill(Jug3)

Pour(Jug3 → Jug4)

Empty(Jug4)

Pour(Jug3 → Jug4)

IF Jug4 == 2 THEN

PRINT "Goal Reached"

ENDIF

STOP

9. Applications

The Water Jug Problem is used in:

- Artificial Intelligence
- State Space Search
- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Robot Planning
- Automated Problem Solving
- Decision Making Systems

10. Conclusion

The Water Jug Problem is a fundamental AI search problem that illustrates how an intelligent agent explores a state space to reach a goal state using valid operations. By systematically applying fill, empty, and pour actions, the agent successfully measures exactly 2 gallons of water in the 4-gallon jug. This problem demonstrates the practical application of search algorithms such as Breadth-First Search (BFS) and Depth-First Search (DFS) in artificial intelligence.

Question 2: Mars Rover – Analyze and Explore Samples on Mars (10 Marks)

Introduction

A **Mars Rover** is an intelligent robotic vehicle designed by space agencies such as NASA to explore the surface of Mars. It operates autonomously or with limited human control to analyze rocks, soil, atmosphere, and search for signs of past or present life. Since communication between Earth and Mars has a significant time delay, the rover must make many decisions independently using Artificial Intelligence (AI).

Examples of Mars rovers include **Sojourner, Spirit, Opportunity, Curiosity,** and **Perseverance**.

i. What are the Percepts of this Agent? (2 Marks)

Definition

Percepts are the information or observations that an intelligent agent receives from its environment through sensors.

Percepts of a Mars Rover

The Mars Rover gathers information using various sensors, such as:

- Images from cameras (terrain, rocks, obstacles)
- Distance to nearby objects (using LiDAR or ultrasonic sensors)
- Soil composition data
- Rock composition and mineral analysis
- Atmospheric conditions (temperature, pressure, humidity)
- Weather information (wind speed and dust storms)
- GPS-like localization and position estimation
- Wheel status and battery level
- Radiation levels
- Surface slope and terrain characteristics

Table: Sensors and Percepts

Sensor	Percept Received
Camera	Images of rocks, terrain, obstacles
Spectrometer	Chemical composition of rocks and soil
Temperature Sensor	Surface temperature
Pressure Sensor	Atmospheric pressure
Proximity Sensor	Distance to obstacles
Navigation Sensor	Position and direction
Battery Sensor	Battery status
Weather Sensor	Wind speed and dust conditions

---

ii. Characterize the Operating Environment (2 Marks)

The Mars Rover operates in a challenging and uncertain environment.

Operating Environment Characteristics

Characteristic	Description
Observable	**Partially Observable** – The rover cannot observe the entire environment at once.
Agents	**Single Agent** – Only the Mars Rover performs the exploration task.
Determinism	**Stochastic** – Outcomes may be affected by dust storms, wheel slippage, or uneven terrain.
Episodes	**Sequential** – Current actions influence future states and decisions.
Dynamics	**Dynamic** – Environmental conditions such as weather and terrain can change.
State Space	**Continuous** – The rover moves through continuous space with varying positions and directions.
Knowledge	**Partially Known** – Some information is known from satellite images, but many regions are unexplored.

Summary

The Mars Rover operates in a **partially observable, stochastic, sequential, dynamic, continuous, single-agent, and partially known environment**.

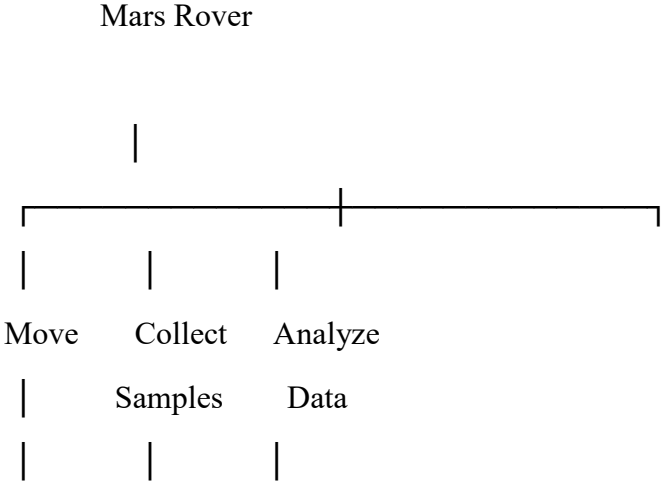
iii. What Actions Can the Agent Take? (2 Marks)

The Mars Rover performs several actions to achieve its mission objectives.

Possible Actions

- Move forward
- Move backward
- Turn left
- Turn right
- Stop
- Avoid obstacles
- Capture images
- Analyze rocks
- Drill into rocks
- Collect soil samples
- Store collected samples
- Measure atmospheric conditions
- Transmit data to Earth
- Recharge using solar panels (for solar-powered rovers)
- Enter power-saving mode
- Plan a safe navigation route

Action Diagram



Avoid Store Send to Earth

Obstacle Samples

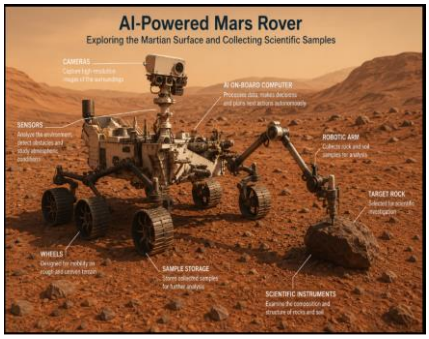
iv. How Can One Evaluate the Performance of the Agent? (2 Marks)

Performance Measure

A **performance measure** evaluates how effectively the Mars Rover achieves its goals.

Performance Criteria

- Successfully reaches the destination Collects maximum scientific sample
- Accurately analyzes rocks and soil
- Avoids obstacles safely
- Conserves battery power
- Completes assigned tasks within the mission time
- Sends accurate data back to Earth
- Minimizes equipment damage
- Operates reliably under harsh environmental conditions



Performance Evaluation Table

Performance Measure Desired Outcome

Sample Collection	-	Maximum samples collected
Navigation	-	Safe path with no collisions
Battery Usage	-	Efficient power consumption
Communication	-	Accurate data transmission
Scientific Analysis	-	High-quality experimental results
Mission Completion	-	All assigned tasks completed

---

v. Suitable Agent Architecture and Justification

Recommended Agent Architecture: Utility-Based Learning Agent

A Utility-Based Learning Agent is the most suitable architecture for a Mars Rover because it not only learns from its environment but also selects actions that maximize mission success while minimizing risks

.

Components of the Architecture

Environment

|

Sensors

|

Perception

|

Learning Element

|

Utility-Based Decision

|

Problem-Solving Module

|

Actuators

|

Environment

Why Utility-Based Learning Agent?

- Learns from previous experiences.
- Makes intelligent decisions under uncertainty.
- Chooses actions that maximize scientific value.
- Conserves battery and other resources.
- Adapts to new terrain and unexpected obstacles.
- Improves navigation over time.
- Handles changing environmental conditions effectively.
- Balances multiple goals, such as safety, energy efficiency, and scientific exploration.

Advantages

- High autonomy
- Adaptive behavior
- Efficient decision-making
- Better resource management
- Reliable performance in unknown environments

Simple Pseudocode

```
START
Initialize sensors and battery
WHILE mission is not complete DO
   Sense the environment
   Capture images
   Detect obstacles
   IF obstacle exists THEN
      Find alternate safe path
   ELSE
      Move forward
   ENDIF
   IF interesting rock is detected THEN
      Analyze rock
      Collect sample
      Store sample
   ENDIF
   Record environmental data
 Send collected information to Earth
   Monitor battery level
END WHILE
STOP
```

Applications of Mars Rover AI

- Space exploration
- Planetary research
- Geological analysis
- Search for signs of life
- Autonomous robotic navigation
- Scientific data collection
- Environmental monitoring

Conclusion

The Mars Rover is an advanced intelligent agent that explores the Martian surface using AI. It perceives its surroundings through sensors, performs autonomous navigation, collects and analyzes samples, and communicates scientific data to Earth. Since it operates in a partially observable, dynamic, and uncertain environment, a Utility-Based Learning Agent architecture is the most appropriate because it enables the rover to adapt, make optimal decisions, and maximize mission success while ensuring safety and efficient resource utilization.

Question 3: Explore the Search Space Using Search Strategies and Formulate the Problem Components for the 8-Queens Problem

1. Introduction

The 8-Queens Problem is a well-known problem in Artificial Intelligence (AI) that demonstrates state-space search and constraint satisfaction. The objective is to place eight queens on an 8 × 8 chessboard such that no two queens attack each other.

A queen in chess can move:

- Horizontally
- Vertically
- Diagonally

Therefore, no two queens should be in the same row, same column, or same diagonal.

2. Problem Statement

Place 8 queens on an 8 × 8 chessboard so that:

- No two queens are in the same row.
- No two queens are in the same column.
- No two queens are on the same diagonal.

The given configuration is not a solution because the queen in the first column lies on the same diagonal as the queen in the last column.

Python code implementation

ai.py - C:/Users/M. Sharmila/OneDrive/Documents/ai.py (3.14.2)

File Edit Format Run Options Window Help

```
N = 8

board = [[0 for _ in range(N)] for _ in range(N)]

def isSafe(board, row, col):

    for i in range(col):
        if board[row][i] == 1:
            return False

    i = row
    j = col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    i = row
    j = col
    while i < N and j >= 0:
        if board[i][j] == 1:
            return False
        i += 1
        j -= 1

    return True

def solve(board, col):

    if col >= N:
        return True

    for i in range(N):

        if isSafe(board, i, col):

            board[i][col] = 1

            if solve(board, col + 1):
                return True

            board[i][col] = 0

    return False
```

OUTCOME

```
========== RESTART:
[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]
```

3. Problem Formulation

a) Initial State

The chessboard is empty.

No queens are placed.

b) State

A **state** represents the arrangement of queens on the chessboard.

Example:

State = (1,5,8,6,3,7,2,4)

Each number represents the row position of the queen in its respective column.

For example:

Column	Row
1	1
2	5
3	8
4	6
5	3
6	7
7	2
8	4

c) Initial State Representation

□□□□□□□□

□□□□□□□□

□□□□□□□□

□□□□□□□□

□□□□□□□□

□□□□□□□□

□□□□□□□□

□□□□□□□□

(No queens placed)

d) Operators (Actions)

The agent can perform the following action:

- Place a queen in a safe position.
- Move a queen to another row.
- Backtrack if a conflict occurs.

e) Goal State

A configuration where:

- All 8 queens are placed.
- No two queens attack each other.

Example of a valid solution:

Q . . . . . .

. . . . Q . . .

. . . . Q

. . . . . Q . .

. . Q . . . . .

. . . . . Q .

. Q . . . . . .

. . . Q . . . .

4. State Space

The **state space** is the collection of all possible arrangements of queens on the board.

If every square is considered,

$64 \times 63 \times 62 \times ...$

possible arrangements exist.

However, since only **one queen is placed in each column**, the state space reduces significantly.

The search algorithm explores only the valid states.

5. Constraints

The following constraints must always be satisfied.

Constraint 1

No two queens should be in the same row.

Example (Invalid)

Q Q . . . . . .

Constraint 2

No two queens should be in the same column.

Example (Invalid)

Q

Q

Constraint 3

No two queens should be on the same diagonal.

Example (Invalid)

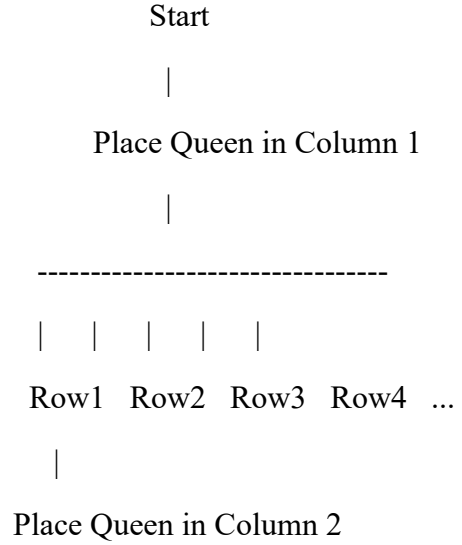
Q . . .

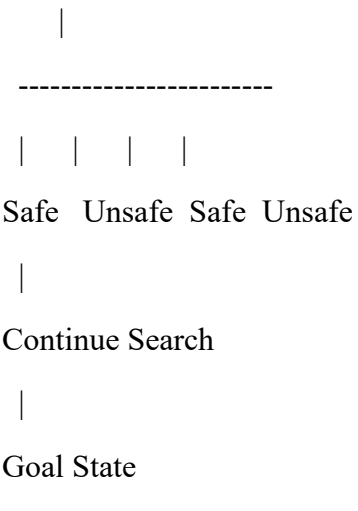
. Q . .

. . Q .

. . . Q

6. Search Space Tree





The search continues until all eight queens are placed successfully.

7. Search Strategies Used

The 8-Queens problem can be solved using different AI search strategies.

a) Depth-First Search (DFS)

- Places queens one column at a time.
- Continues until a conflict occurs.
- Uses backtracking.
- Requires less memory.

b) Breadth-First Search (BFS)

- Explores all possible states level by level.
- Finds a solution but requires large memory.

c) Backtracking Search (Most Common)

- Places queens one by one.
- If a conflict occurs, returns to the previous queen.
- Tries another position.

This is the most efficient and widely used method.

d) Heuristic Search

Uses heuristic functions to reduce unnecessary exploration and reach the solution faster.

8. Backtracking Algorithm

Algorithm

Step 1: Start.

Step 2: Place the first queen in Column 1.

Step 3: Move to the next column.

Step 4: Check whether the position is safe.

Step 5: If safe, place the queen.

Step 6: Otherwise, try the next row.

Step 7: If no row is safe,

backtrack to the previous column.

Step 8: Repeat until all eight queens are placed.

Step 9: Display the solution.

Step 10: Stop.

9. Pseudocode

```
procedure Solve(col)
if col > 8
   print Solution
   return TRUE
for each row from 1 to 8
   if Safe(row, col)
      Place Queen(row, col)
      if Solve(col + 1)
         return TRUE
      Remove Queen(row, col)
return FALSE
```

12. Applications of the 8-Queens Problem

- Artificial Intelligence (AI)
- Constraint Satisfaction Problems (CSP)
- Backtracking Algorithms
- Robotics Path Planning

- Task Scheduling
- Resource Allocation
- Optimization Problems
- Game Development

13. Advantages

- Demonstrates state-space search techniques.
- Introduces constraint satisfaction concepts.
- Efficiently solved using backtracking.
- Helps understand recursive algorithms.
- Widely used in AI education and research.

14. Conclusion

The 8-Queens Problem is a classic Constraint Satisfaction Problem (CSP) in Artificial Intelligence. It involves placing eight queens on a chessboard such that no two queens threaten each other. By formulating the problem with an initial state, state space, operators, constraints, and goal state, AI search strategies such as Depth-First Search (DFS) and Backtracking can efficiently find a valid solution. Due to its simplicity and effectiveness, the 8-Queens Problem remains an important example for studying search algorithms, recursion, and optimization techniques in AI.

Question 4: A Customer Wants to Travel Using the OLA Cab Booking Mobile Application – Identify the Problem-Solving Agent and Write the Pseudocode

1. Introduction

An OLA Cab Booking System is an example of an Artificial Intelligence (AI)-based Problem-Solving Agent. It helps customers book a suitable cab by analyzing user requirements such as pickup location, destination, cab type, fare, availability, and estimated arrival time.

The intelligent agent perceives the environment, processes the information, selects the most appropriate cab, and performs the booking to achieve the user's goal.

2. Problem Statement

A customer wants to travel from one location to another using the OLA Cab Booking mobile application. The customer can choose from different cab types such as:

- Mini
- Micro
- Sedan
- Prime
- Share

The AI agent should identify the best available cab based on the customer's preferences and complete the booking.

3. Problem Formulation

Initial State

- Customer opens the OLA application.
- No cab is booked.

State Space

The state space includes all possible booking options available to the customer.

Example:

- Different cab types
- Different drivers

- Various routes
- Different fare prices
- Estimated arrival times

Actions (Operators)

The intelligent agent can perform the following actions:

- Detect current location.
- Accept destination.
- Display available cab types.
- Calculate fare.
- Estimate arrival time.
- Search for nearby drivers.
- Allocate the nearest driver.
- Confirm booking.
- Track the ride.
- Complete payment.

Goal State

The customer successfully reaches the destination after completing the ride.

4. PEAS Representation

PEAS (Performance Measure, Environment, Actuators, Sensors) is used to describe an intelligent agent.

Component	**Description**
Performance Measure	Fast booking, minimum waiting time, shortest route, low fare, customer satisfaction
Environment	Roads, GPS, traffic, drivers, passengers, weather conditions
Actuators	Mobile screen, booking confirmation, navigation system, notifications

Component	Description
Sensors	GPS, internet, customer location, destination input, traffic data

5. Type of Intelligent Agent

The **Goal-Based Agent** is the most suitable intelligent agent for the OLA Cab Booking System.

Reason

A Goal-Based Agent selects actions that help achieve a specific goal.

The goal is:

Transport the customer safely from the pickup location to the destination with minimum cost and waiting time.

The agent continuously evaluates different options before making the booking.

6. Working of the Goal-Based Agent

Step 1

Customer enters:

- Pickup location
- Destination

↓

Step 2

Agent collects information:

- GPS location
- Nearby drivers
- Traffic condition
- Available cab types

↓

Step 3

Agent calculates:

- Estimated fare
- Distance
- Estimated arrival time

↓

Step 4

Agent displays:

- Mini
- Micro
- Sedan
- Prime
- Share

↓

Step 5

Customer selects the preferred cab.

↓

Step 6

Agent assigns the nearest available driver.

↓

Step 7

Driver reaches the pickup location.

↓

Step 8

Trip starts.

↓

Step 9

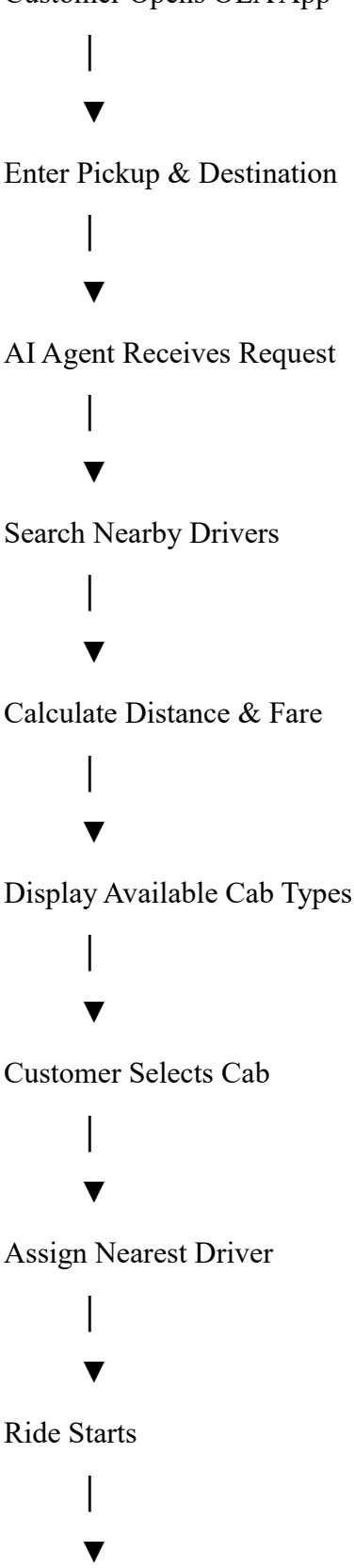
Customer reaches the destination.

↓

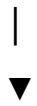
Step 10

Payment is completed.

7. Flow Diagram



Reach Destination



Payment Completed

8. Algorithm

Algorithm: OLA Cab Booking

1. Start.
2. Open the OLA application.
3. Enter the pickup location.
4. Enter the destination.
5. Detect the customer's current location using GPS.
6. Search for nearby available drivers.
7. Calculate the distance, estimated fare, and arrival time.
8. Display available cab types (Mini, Micro, Sedan, Prime, Share).
9. Customer selects a preferred cab.
10. Assign the nearest available driver.
11. Confirm the booking.
12. Track the ride using GPS.
13. Complete the ride and process payment.
14. Stop.

9. Pseudocode

START

Input Pickup_Location

Input Destination

Detect Current Location using GPS

Search Available Driver

```
Calculate Distance
Calculate Fare
Display Cab Types
Input Selected_Cab
IF Driver Available THEN
    Assign Driver
    Confirm Booking
    Start Ride
    Track Ride
    Complete Payment
    Print "Trip Completed Successfully"
ELSE
    Print "No Cab Available"

ENDIF
STOP
```

CODE IMPLEMENTATION

ai.py - C:/Users/M. Sharmila/OneDrive/Documents/ai.py (3.14.2)

File Edit Format Run Options Window Help

```
print("***** OLA CAB BOOKING SYSTEM *****")

pickup = input("Enter Pickup Location: ")
destination = input("Enter Destination: ")

cab_types = ["Mini", "Micro", "Sedan", "Prime", "Share"]

print("\nAvailable Cab Types:")
for i, cab in enumerate(cab_types, start=1):
    print(f"{i}. {cab}")

choice = int(input("\nSelect Cab (1-5): "))

selected_cab = cab_types[choice - 1]

print("\nSearching for nearby drivers...")

driver_available = True

if driver_available:
    print("Driver Assigned Successfully")
    print("Selected Cab :", selected_cab)
    print("Pickup :", pickup)
    print("Destination :", destination)
    print("Booking Confirmed")
    print("Trip Started...")
    print("Trip Completed Successfully")
    print("Payment Successful")
else:
    print("Sorry! No Drivers Available.")
```

OUTPUT

```
Select Cab (1-5): 5

Searching for nearby drivers...
Driver Assigned Successfully
Selected Cab : Share
Pickup : kundrathur
Destination : anakaputhur
Booking Confirmed
Trip Started...
Trip Completed Successfully
Payment Successful
>> 100
100
```

12. Advantages

- Easy and convenient booking.
- Saves travel time.

- Displays multiple cab options.
- Provides estimated fare before booking.
- Tracks the ride in real time using GPS.
- Ensures secure online payment.
- Improves customer satisfaction through efficient service.

13. Applications

- OLA
- Uber
- Rapido
- Taxi booking services
- Logistics and transportation systems
- Ride-sharing platforms

14. Conclusion

The OLA Cab Booking System is a practical application of an AI Goal-Based Agent. The agent collects information such as the customer's location, destination, nearby drivers, traffic conditions, and cab availability, then determines the best cab option to achieve the user's goal. By using search strategies and decision-making, the system provides an efficient, cost-effective, and user-friendly transportation service.

Question 5: A Logistics Company Operates a Delivery Network Where Packages Must Be Transported Between Warehouses at Minimum Cost. Each Route Between Warehouses Has an Associated Cost. Formulate the Problem Using AI Search Techniques and Write the Algorithm/Pseudocode.

1. Introduction

A Logistics Network is a transportation system that moves packages from one warehouse to another through interconnected routes. In Artificial Intelligence (AI), this problem is modeled as a Shortest Path Search Problem, where the objective is to determine the least-cost route between a source warehouse and a destination warehouse.

AI search algorithms such as Uniform Cost Search (UCS), Dijkstra's Algorithm, and A* Search are commonly used to solve this problem efficiently.

2. Problem Statement

A logistics company operates multiple warehouses connected by transportation routes. Each route has a specific transportation cost. The objective is to transport packages from a source warehouse to a destination warehouse while minimizing the total transportation cost.

3. Problem Formulation

Initial State

- Package is at the source warehouse.

Example:

Warehouse A

State Space

The state space consists of all warehouses and the possible routes connecting them.

Example:

A → B → C → D → E

Each warehouse represents a state.

Actions (Operators)

The AI agent can perform the following actions:

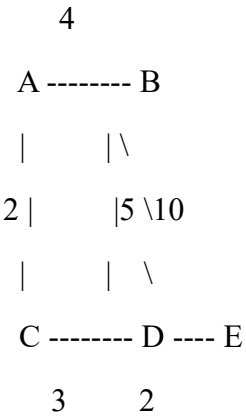
- Move package from one warehouse to another.
- Calculate transportation cost.
- Compare different routes.
- Select the least-cost path.
- Deliver the package.

Goal State

The package reaches the destination warehouse with the minimum transportation cost.

4. Search Space Representation

Consider the following logistics network.



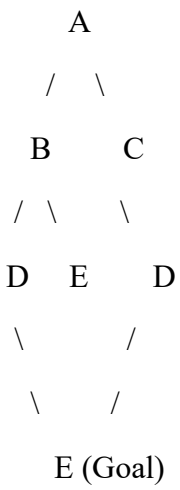
Where:

- A = Source Warehouse
- E = Destination Warehouse

Edge costs:

- A → B = 4
- A → C = 2
- C → D = 3
- B → D = 5
- B → E = 10
- D → E = 2

5. State Space Tree



The search algorithm explores all possible routes and selects the one with the minimum total cost.

6. Search Strategy Used

Uniform Cost Search (UCS)

Uniform Cost Search expands the node with the lowest cumulative path cost.

Characteristics:

- Finds the optimal solution.
- Guarantees minimum transportation cost.
- Suitable when route costs are different.

Alternative Algorithms

- Dijkstra's Algorithm
- A* Search Algorithm
- Breadth-First Search (when all edge costs are equal)

Among these, Uniform Cost Search (UCS) is the most appropriate because it always chooses the least-cost path.

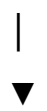
7. Working of the AI Agent

1. Receive source and destination warehouses.
2. Generate all possible routes.

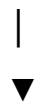
3. Calculate the total transportation cost for each route.
4. Compare all route costs.
5. Select the route with the minimum cost.
6. Deliver the package.
7. Display the optimal path and total cost.

8**. Flow Diagram**

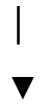
Start



Input Source Warehouse



Input Destination Warehouse



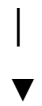
Generate All Possible Routes

Calculate Route Cost

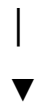
Compare Costs

Choose Minimum Cost Route

Deliver Package



Display Total Cost



Stop

9. Algorithm

Algorithm: Minimum Cost Logistics Routing

1. Start.
2. Input source warehouse.
3. Input destination warehouse.
4. Read all available routes and transportation costs.
5. Initialize the source node with cost 0.
6. Explore neighboring warehouses.
7. Calculate the cumulative cost for each route.
8. Select the warehouse with the minimum cumulative cost.
9. Repeat until the destination warehouse is reached.
10. Display the shortest path and minimum transportation cost.
11. Stop.

10. Pseudocode

START

Input Source

Input Destination

Initialize Cost(Source) = 0

Place Source in Priority Queue

WHILE Queue is not Empty

Remove Warehouse with Minimum Cost

```
    IF Destination Reached THEN
        Display Shortest Path
        Display Minimum Cost
        STOP
    ENDIF
    FOR each Neighbor Warehouse
        Calculate New Cost
        IF New Cost is Smaller THEN
            Update Cost
            Insert Neighbor into Queue
        ENDIF
    ENDFOR
ENDWHILE
STOP
```

Python code implementation

ai.py - C:/Users/M. Sharmila/OneDrive/Documents/ai.py (3.14.2)

File Edit Format Run Options Window Help

```
import heapq

graph = {
    'A': [('B', 4), ('C', 2)],
    'B': [('D', 5), ('E', 10)],
    'C': [('D', 3)],
    'D': [('E', 2)],
    'E': []
}

def dijkstra(graph, start):
    distances = {node: float('inf') for node in graph}
    distances[start] = 0

    priority_queue = [(0, start)]

    while priority_queue:
        current_distance, current_node = heapq.heappop(priority_queue)

        for neighbor, weight in graph[current_node]:
            distance = current_distance + weight

            if distance < distances[neighbor]:
                distances[neighbor] = distance
                heapq.heappush(priority_queue, (distance, neighbor))

    return distances

source = 'A'
distances = dijkstra(graph, source)

print("Minimum Cost from Source Warehouse:")
for warehouse in distances:
    print(source, "->", warehouse, "=", distances[warehouse])
```

OUTPUT:

```
Minimum Cost from Source Warehouse:
A -> A = 0
A -> B = 4
A -> C = 2
A -> D = 5
A -> E = 7
```

13. Advantages

- Finds the minimum transportation cost.
- Reduces delivery time.
- Optimizes route selection.
- Improves fuel efficiency.

- Suitable for large logistics networks.
- Ensures efficient resource utilization.

14. Applications

- Courier services
- E-commerce delivery systems
- Supply chain management
- Warehouse management
- Food delivery services
- Postal services
- Transportation and distribution networks

15. Conclusion

The logistics routing problem is a cost optimization problem in Artificial Intelligence. By representing warehouses as nodes and transportation routes as weighted edges, AI search algorithms such as Uniform Cost Search (UCS) and Dijkstra's Algorithm can identify the least-cost path between the source and destination. This approach minimizes transportation costs, improves delivery efficiency, and supports effective decision-making in logistics and supply chain management.